

REPORTE TÉCNICO

Proyecto Final — Aprendizaje Profundo

Clasificación de Reseñas de Hoteles Andaluces con Redes Neuronales Recurrentes

Felipe Reyes – Yibby Gonzalez – Sebastian Ruiz – Daniel Ruiz

Pontificia Universidad Javeriana · Maestría en Inteligencia Artificial · 2026

Profesor: Ing. Julio Omar Palacio Niño, M.Sc.

Mayo 2026

1. Objetivo del Proyecto

El presente reporte técnico documenta el desarrollo completo del Proyecto Principal de la asignatura de Aprendizaje Profundo. El objetivo es construir, entrenar y comparar múltiples arquitecturas de redes neuronales recurrentes (RNN) para la clasificación de sentimiento en reseñas de hoteles en español, utilizando el dataset Andalusian Hotels Reviews.

El problema se define como una tarea de clasificación multiclase de texto en cinco categorías correspondientes a calificaciones de 1 a 5 estrellas. Se implementaron dos modelos clásicos (LSTM unidireccional y BiLSTM bidireccional) y un modelo nuevo/combinado con mecanismos de atención (BiLSTM + Multi-Head Attention), todos entrenados y evaluados bajo las mismas condiciones experimentales para garantizar comparabilidad.

2. Análisis del Dataset y Dimensionalidad

2.1 Descripción General

El dataset Andalusian Hotels Reviews (Unbalanced) contiene reseñas de hoteles de Andalucía, España, escritas en español. Cada reseña está etiquetada con una calificación de 1 a 5 estrellas, representadas internamente como etiquetas enteras 0–4 en PyTorch. El dataset presenta un desbalance severo con ratio 9.1:1 entre la clase mayoritaria (5) y la minoritaria (2).

Característica	Valor
Fuente	Kaggle: andalusian-hotels-reviews-unbalanced
Total de muestras	18,172
Columnas relevantes	review_text (texto), label (0–4)
Idioma	Español
Train set	12,720 muestras (70%)
Validación	2,726 muestras (15%)
Test set	2,726 muestras (15%)
Estrategia de split	Estratificado, semilla fija = 42
Limitación conocida	2,674 duplicados en train (21%); solapamiento train/val=868, train/test=902

Tabla 1. Descripción dataset.

Limitación importante: se detectaron duplicados exactos en el train set (21%) y solapamiento entre splits. Esto constituye un data leakage leve que puede inflar ligeramente las métricas reportadas. Se documenta como limitación del experimento.

2.2 Distribución de Clases

El dataset presenta un desbalance marcado que afecta directamente el diseño de los modelos. La clase 5 concentra casi la mitad del corpus de entrenamiento, lo que hace que un clasificador trivial que siempre prediga la clase mayoritaria obtendría ~49.6% de accuracy sin aprender nada útil.

Clase	Estrellas	Muestras train	% train	Class weight (balanced)
0	1	1,174	9.2%	2.167
1	2	696	5.5%	3.655
2	3	1,592	12.5%	1.598
3	4	2,955	23.2%	0.861
4	5	6,303	49.6%	0.404

Tabla 2. Distribución de clases en el dataset (conteo y porcentaje train).



Figura 1. Distribución de clases en el dataset (conteo y porcentaje por split).

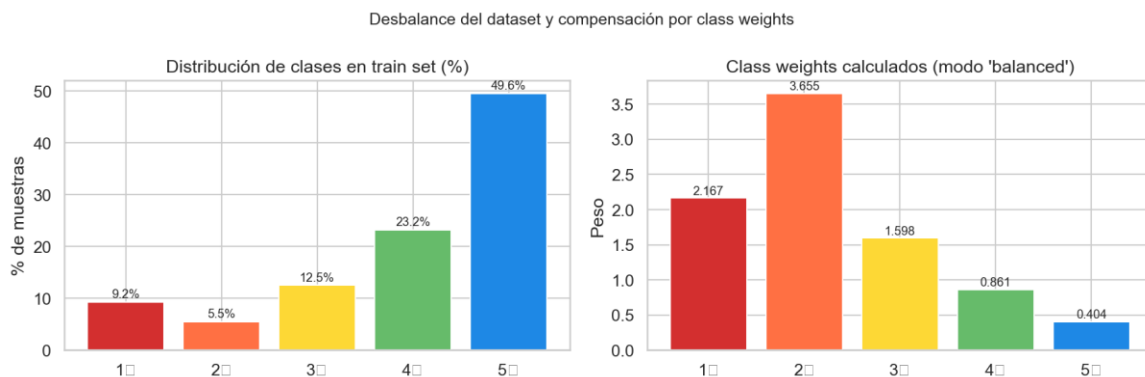


Figura 2. Distribución de clases en train set y class weights calculados (modo balanced). Las clases minoritarias reciben mayor peso para compensar el desbalance.

Al contrastar la distribución por splits (**Figura 1**) se evidencia que el proceso de partición fue estrictamente estratificado, manteniendo la misma proporción de sesgo hacia calificaciones excelentes en los tres subconjuntos. Sin embargo, para evitar que las redes neuronales recurrentes se sesguen de forma perezosa hacia la clase mayoritaria (calificación 5), se implementó la técnica de pesos de

clase inversamente proporcionales a su frecuencia (**Figura 2**). Al penalizar con mayor severidad los errores en las clases más escasas (como la calificación 2, que recibe un peso de \$3.655\$), se obliga a la función de pérdida a prestar especial atención a los patrones lingüísticos de las reseñas menos frecuentes, garantizando que el modelo final aprenda a generalizar la insatisfacción y la neutralidad de los usuarios, y no solo la polaridad positiva dominante.

2.3 Análisis de Longitud de Reseñas

El análisis de longitud determinó el parámetro MAX_LEN del pipeline. Se midió tanto en caracteres como en tokens (palabras tras limpieza con clean_text).

Estadística	Caracteres	Tokens
Media	456	78.5
Desviación estándar	307.2	54.9
Mínimo	61	7
Percentil 25	250	42
Mediana (P50)	355	61
Percentil 75	590	102
Percentil 95 (MAX_LEN base)	790	148
Máximo	7,777	1,416

Tabla 3. Descriptivas de longitud de reseñas.

El percentil 95 en tokens es 148. Se definió MAX_LEN=150 para el LSTM (Felipe) y MAX_LEN=200 para el BiLSTM y el modelo combinado, cubriendo respectivamente el ~95% y ~97-98% de todas las reseñas.

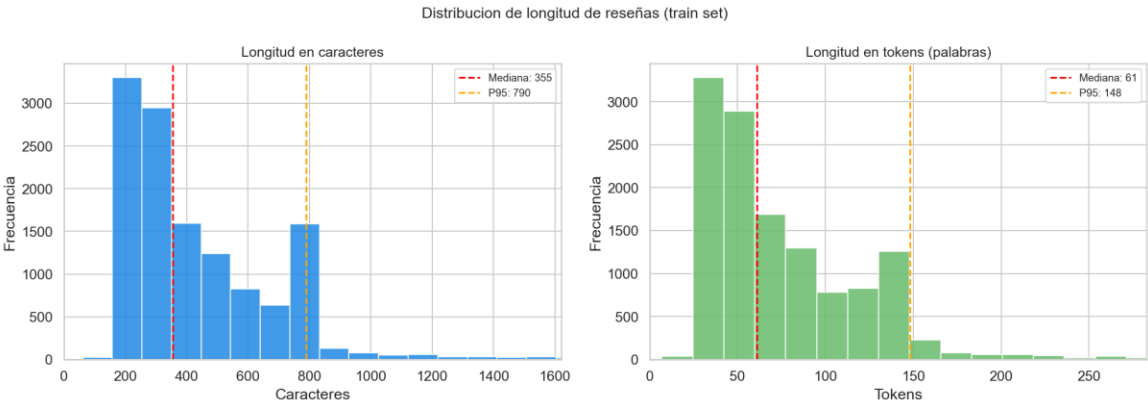


Figura 3. Histograma de longitudes de reseñas en caracteres y tokens. La mayoría de reseñas son cortas (media 78.5 tokens).

El comportamiento asimétrico observado en el histograma (**Figura 3**) justifica técnicamente la elección de **MAX_LEN=150 y MAX_LEN=200**. Al truncar los textos en estos umbrales, se logra un balance óptimo: se conserva la información semántica del grueso del corpus (cubriendo hasta el 98% de los datos) mientras se reduce drásticamente el costo computacional y se evita el desvanecimiento del gradiente (vanishing gradient) al procesar secuencias innecesariamente largas llenas de padding. Por otro lado, el diagrama de caja (**Figura 4**) revela un fenómeno conductual interesante: las reseñas de insatisfacción extrema (clase 1) tienden a ser las más largas (mediana 74, media 92.2 tokens), ya que los clientes detallan sus quejas. En contraste, las reseñas de satisfacción absoluta (clase 5) son

las más cortas y directas (mediana 58, media 73.1 tokens). Las clases intermedias (2-4) presentan longitudes similares entre sí. Este hallazgo indica que las secuencias más largas concentran información de sentimiento negativo, un reto que las redes BiLSTM y combinadas están mejor preparadas para capturar debido a su contexto bidireccional.

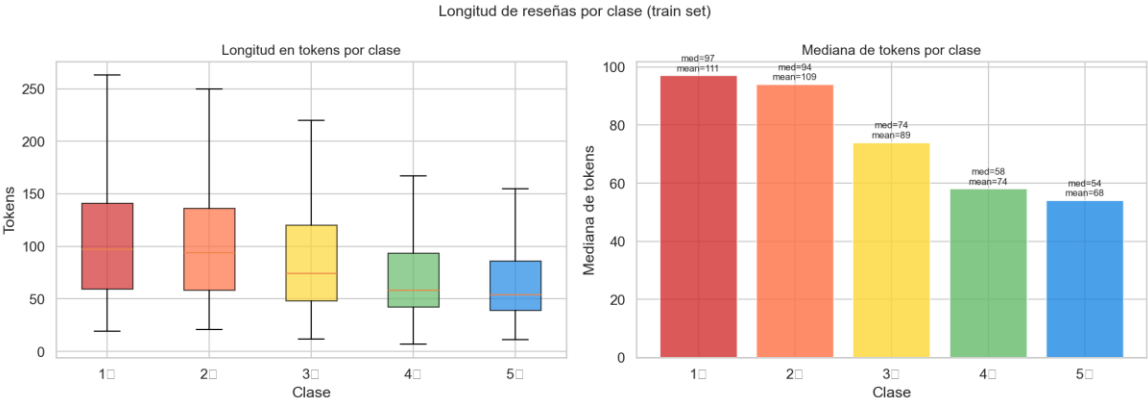


Figura 4. Boxplot de longitudes por clase. Las clases extremas (1 y 5) tienden a reseñas más cortas que las intermedias.

2.4 Vocabulario

Tipo de vocabulario	Tokens únicos	Observación
Bruto (sin limpieza)	57,231	Incluye puntuación, mayúsculas, variantes ortográficas
Lowercase sin puntuación	52,449	Normalización básica
Limpio (con clean_text)	27,967	Reducción del 51.1% respecto al bruto
VOCAB_SIZE usado (con PAD+UNK)	20,000	Cubre el 99.2% de todos los tokens del corpus limpio

Tabla 4. Conteo de tokens únicos y porcentaje de reducción en las fases de preprocesamiento.

El análisis cuantitativo expuesto en la **Tabla 3** permite observar cómo el proceso de limpieza reduce el ruido en los textos de forma eficiente. Al remover la puntuación, las mayúsculas y las variantes ortográficas del lenguaje informal, el total de palabras únicas disminuyó drásticamente de 57,231 a 27,967, lo que significa que se eliminó más de la mitad del vocabulario redundante o ruidoso.

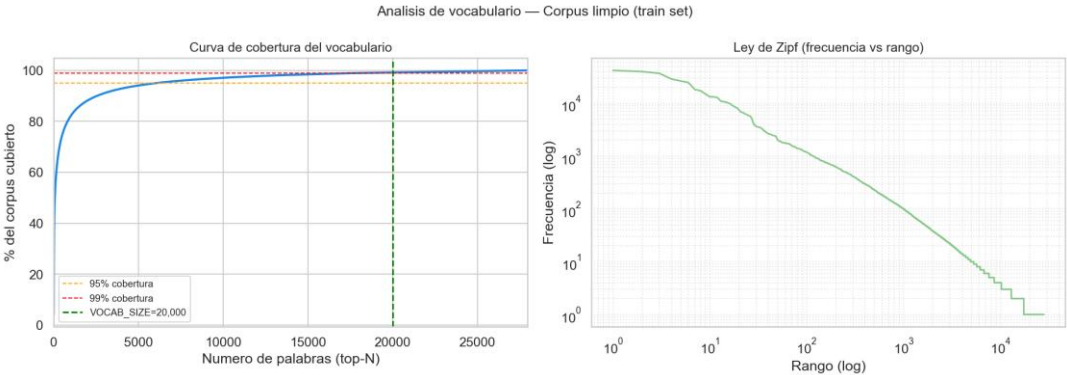


Figura 5. Distribución Zipf del vocabulario y curva de cobertura acumulada. El top-20,000 palabras cubre el 99.2% de todos los tokens.

La Figura 5 demuestra visualmente por qué fijar el tamaño del vocabulario en 20,000 palabras (VOCAB_SIZE) es la mejor decisión metodológica para los modelos del equipo. En la gráfica azul de la

izquierda (Curva de cobertura), se observa que a medida que incluimos los términos más repetidos, la cobertura del texto sube muy rápido; al llegar a la línea verde de 20,000 términos, ya se logra conservar el 99.2% de las palabras más importantes de todo el corpus limpio.

Por su parte, la gráfica verde de la derecha (Ley de Zipf) confirma que el idioma de las reseñas sigue un comportamiento natural: unas pocas palabras clave se repiten constantemente (zona alta de la curva), mientras que miles de palabras extrañas o con errores de digitación aparecen una sola vez (la caída de la línea). Al recortar el diccionario en este umbral óptimo, se evita que las redes neuronales como la LSTM y la BiLSTM gasten memoria procesando tokens irrelevantes, obligando a las capas de embeddings a concentrar su aprendizaje únicamente en los términos más significativos del dataset.

2.5 Análisis Léxico por Clase

2.5.1 Palabras más frecuentes

Las 10 palabras más frecuentes son stop words (de, y, la, el, que...) con frecuencias de 12,000–41,000. El primer término de dominio relevante es 'hotel' (~9,876 ocurrencias). Para las visualizaciones del EDA se eliminaron las stop words para mostrar vocabulario de sentimiento más informativo; en el pipeline de entrenamiento se conservan (ver sección 3.1).

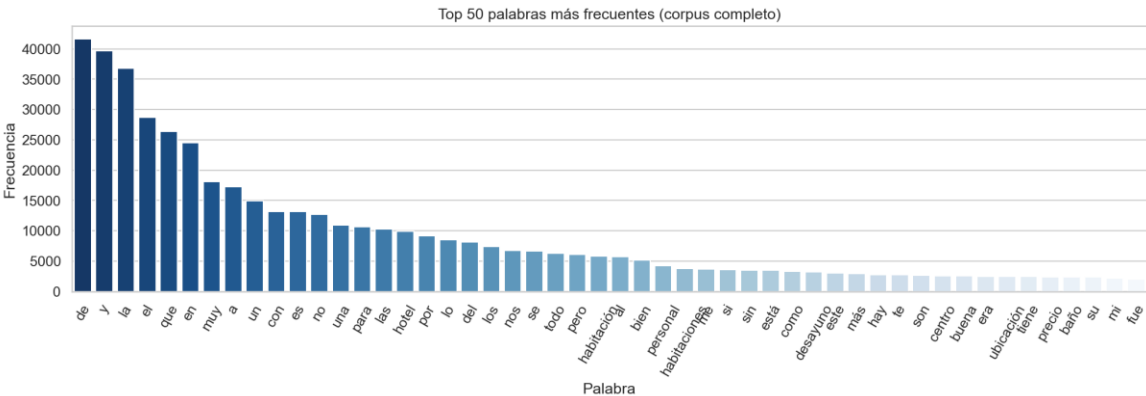


Figura 6. Top 50 palabras más frecuentes en el corpus completo (con stop words).

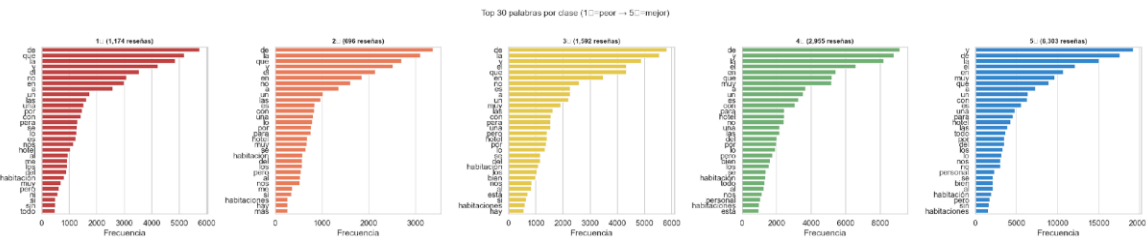


Figura 7. Top 30 palabras más frecuentes por clase (sin stop words). Las clases extremas muestran vocabulario claramente diferenciado.

La Figura 7 permite observar las 30 palabras que más se repiten en el dataset según la calificación en estrellas de los usuarios. El hallazgo principal de esta gráfica es que, sin importar si la reseña es muy mala (clase 1 en rojo) o excelente (clase 5 en azul), las palabras que dominan los primeros lugares son exactamente las mismas: conectores y artículos como "de", "que", "la", "y", "el".

Este comportamiento visual confirma que las palabras con mayor frecuencia estadística en el lenguaje natural no son necesariamente las que tienen mayor valor informativo. Si se entrenaran los modelos recurrentes dejando estos términos, las redes neuronales se confundirían al encontrar las mismas palabras en todas las categorías. Esto demuestra la necesidad estricta de aplicar funciones de filtrado como `clean_text` para eliminar estos conectores comunes y permitir que palabras con verdadera carga

emocional o descriptiva (como "habitación", "personal" o "bien") suban en importancia y guían correctamente la clasificación del modelo.



Figura 8. Nubes de palabras por clase (sin stop words). Clase 1: términos negativos dominantes. Clase 5★: términos positivos.

Las nubes de palabras de la Figura 8 representan visualmente los términos más comunes antes de aplicar la limpieza avanzada. En los cinco gráficos se observa que se observan los términos de sentimiento más relevantes diferenciados por clase: vocabulario negativo (horrible, decepción, sucia) en las clases bajas y vocabulario positivo (excelente, maravilloso, perfecto) en las clases altas. Este resultado visual refuerza que el vocabulario de sentimiento se diferencia claramente entre clases extremas, validando la capacidad discriminativa del texto para la tarea de clasificación.

3. Pipeline de Preprocesamiento

3.1 Módulo src/preprocessing.py

Se implementó un módulo Python reutilizable que encapsula todo el proceso de preparación de datos. La función orquestadora `build_pipeline()` recibe un diccionario de configuración y devuelve `DataLoaders` de PyTorch listos para entrenar, garantizando que todos los modelos usen exactamente el mismo preprocesamiento.

Función	Descripción
<code>clean_text()</code>	Lowercase → normalización NFKD (preserva tildes y ñ) → elimina URLs/emails/números → elimina caracteres especiales → normaliza espacios
<code>build_tokenizer()</code>	Construye vocabulario top-20,000 palabras SOLO desde train. Reserva <PAD>=0, <UNK>=1
<code>encode_sequences()</code>	Convierte texto a índices enteros con padding/truncado a longitud fija
<code>build_splits()</code>	Splits estratificados 70/15/15, semilla=42, guardados como CSV
<code>compute_class_weights()</code>	Pesos balanced ($n_{total} / n_{classes} \times n_c$) para CrossEntropyLoss
<code>ReviewDataset</code>	Dataset PyTorch: devuelve (secuencia, etiqueta) como tensores
<code>build_pipeline()</code>	Orquesta todo y devuelve train/val/test DataLoaders + word2idx + class_weights

Tabla 5. Descripción de funciones componentes del pipeline de preprocesamiento y preparación de datos.

Decisión de diseño	Opción elegida	Justificación
Stop words	CONSERVADAS en modelos	'no' aparece en 87% de reseñas 1★ vs 33% en 5★. Negaciones y conectores son críticos para sentimiento en RNNs secuenciales
Tildes	PRESERVADAS	'mas' ≠ 'más' en español. Eliminarlas introduce ambigüedades léxicas perjudiciales

Decisión de diseño	Opción elegida	Justificación
Tokenización	Por palabras (top-20,000)	Apropiado para LSTM/BiLSTM clásicos. Subwords son para transformers
MAX_LEN	150 (LSTM) / 200 (BiLSTM, Combinado)	Basado en percentil 95 = 148 tokens
Embeddings	Entrenables desde cero	Sin pre-cargados en modelos clásicos; permite comparación limpia entre arquitecturas
Splits	70/15/15 estratificados, semilla 42	Garantiza representación proporcional. Generados una vez por Yibby; todos los demás los cargan

Tabla 6. Matriz de decisiones de diseño en el preprocesamiento y configuración del pipeline.

3.2 Infraestructura de Entrenamiento

Daniel Ruiz desarrolló los módulos `src/training.py` y `src/metrics.py` utilizados por todos los modelos, garantizando condiciones experimentales idénticas y comparabilidad de resultados.

Módulo	Funciones	Descripción
<code>src/training.py</code>	<code>set_seed</code> , <code>EarlyStopping</code> , <code>train</code> , <code>evaluate</code>	Entrenamiento con early stopping, checkpointing del mejor modelo, ReduceLROnPlateau scheduler, gradient clipping
<code>src/metrics.py</code>	<code>compute_metrics</code> , <code>save_metrics_json</code> , <code>plot_training_curves</code> , <code>plot_confusion_matrix</code> , <code>finalize_and_save</code>	Cálculo de métricas, exportación JSON estandarizado, generación de figuras PNG

Tabla 7. Estructura de módulos de la solución para el entrenamiento y la evaluación de modelos.

4. Metodología — Arquitecturas Implementadas

Se implementaron tres arquitecturas para el Proyecto Principal, todas en PyTorch con el mismo pipeline de preprocesamiento y esqueleto de entrenamiento, lo que garantiza comparabilidad directa de resultados.

4.1 Modelo Clásico 1 — LSTM Unidireccional

Se implementó una red LSTM (Long Short-Term Memory) unidireccional como primer modelo clásico. El LSTM procesa la secuencia de izquierda a derecha, manteniendo un estado oculto que captura dependencias de largo alcance. Se tomó el hidden state del último token como representación de toda la reseña.

Capa	Tipo	Dimensiones	Descripción
1	Embedding	20,000 × 128	Entrenables desde cero. <code>padding_idx=0</code> para ignorar tokens de relleno
2	LSTM	128 → 128 (1 capa)	Unidireccional. Se toma el hidden state del último token no-padding

Capa	Tipo	Dimensiones	Descripción
3	Linear + ReLU	128 → 64	Capa de reducción con activación ReLU
4	Dropout	p=0.3	Regularización para prevenir overfitting
5	Linear (logits)	64 → 5	Capa de clasificación final

Tabla 8. Arquitectura detallada y dimensiones de las capas del modelo base LSTM unidireccional.

Se entrenaron 4 versiones explorando distintas tasas de aprendizaje, estrategias de class weights y longitudes de secuencia:

Versión	LR	Class Weights	MAX_LEN	F1 macro	Accuracy	Obs.
v1	1e-3	Balanced	Temporal	0.3127	0.2982	Pipeline temporal; accuracy anómala
v2	3e-4	Ninguno	Temporal	0.3830	0.5675	Sin class weights mejora accuracy
v3 ✓	3e-4	Ninguno	150	0.3996	0.6042	VERSIÓN DEFINITIVA
v4 sqrt	3e-4	Sqrt	150	0.3919	0.5829	Pesos raíz cuadrada no mejoran

Tabla 9. Histórico de experimentos y ajuste de hiperparámetros para la optimización del modelo base LSTM.

Hiperparámetro	Valor (versión definitiva)
Embedding dim	128
Hidden size	128
Dropout	0.3
Learning rate	3e-4
MAX_LEN	150
Class weights	No
Épocas ejecutadas	24
Mejor época	19
Tiempo	321.2 s
Parámetros	2,700,933

Tabla 10. Configuración final de hiperparámetros y métricas de entrenamiento para el modelo definitivo LSTM

4.2 Modelo Clásico 2 — BiLSTM Bidireccional

Se implementó una BiLSTM (Bidirectional LSTM) de 2 capas como segundo modelo clásico. A diferencia del LSTM unidireccional, la BiLSTM procesa cada secuencia simultáneamente en dos direcciones (izquierda→derecha y derecha→izquierda), capturando contexto pasado y futuro para cada token. Esto es especialmente útil en análisis de sentimiento donde modificadores y negaciones pueden aparecer antes o después del término que modifican.

En lugar de usar solo el hidden state del último token, se aplicó Global Max Pooling temporal: toma el valor máximo en cada dimensión a lo largo de toda la secuencia, siendo más robusto para reseñas de longitud variable.

Capa	Tipo	Dimensiones	Descripción
1	Embedding	20,002 × 128	Entrenables. padding_idx=0
2	BiLSTM	128 → 256 (128×2, 2 capas)	Bidireccional. dropout=0.4 entre capas LSTM
3	Global Max Pooling	256 → 256	Máximo en dimensión temporal. Captura rasgos activados en cualquier posición
4	Dropout + Linear + ReLU	256 → 128	Reducción con regularización
5	Linear (logits)	128 → 5	Clasificación final

Tabla 11. Arquitectura detallada y dimensiones de las capas del modelo avanzado BiLSTM profundo.

Versión	Class Weights	F1 macro	Accuracy	Observación
bilstm_v1 ✓	No	0.5749	0.6871	VERSIÓN DEFINITIVA. Mayor accuracy global
bilstm_v2	Sí (balanced)	0.5989	0.6453	Mayor F1 macro (+2.4pp) pero peor accuracy (-4.2pp)

Tabla 12. Histórico de experimentos y evaluación del impacto de pesos de clase en el modelo avanzado BiLSTM.

Hiperparámetro	Valor (versión definitiva)
Embedding dim	128
Hidden size por dirección	128 (256 total)
N° capas BiLSTM	2
Dropout	0.4
Learning rate	1e-3
Weight decay	1e-5
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=2)
Class weights	Sí: [2.167, 3.655, 1.598, 0.861, 0.404]
MAX_LEN	200
Épocas ejecutadas	9

Hiperparámetro	Valor (versión definitiva)
Mejor época	5
Tiempo	27,685.9 s (CPU)
Parámetros	3,253,253

Tabla 13. Configuración final de hiperparámetros y métricas de entrenamiento para el modelo definitivo BiLSTM

4.3 Modelo Nuevo/Combinado — BiLSTM + Multi-Head Attention

Se implementaron tres variantes de arquitecturas que combinan redes recurrentes con mecanismos de atención. La motivación es resolver el 'cuello de botella de representación' de los modelos clásicos: el BiLSTM comprime toda la secuencia en un único vector, lo que puede diluir palabras clave como 'horrible' o 'excelente' en reseñas largas. La atención le asigna a cada token un peso de importancia, permitiendo al modelo enfocarse en las partes más relevantes de cada reseña.

4.3.1 Variante 1 — BiLSTM + Atención de Bahdanau

La atención de Bahdanau (2015) es un mecanismo de atención aditiva que aprende un vector de contexto ponderado. Combina el estado oculto de la BiLSTM con un vector de atención aprendido para calcular scores de importancia por token.

Hiperparámetro	Valor
Celda recurrente	BiLSTM (2 capas)
Hidden size	256 por dirección
Mecanismo atención	Bahdanau (aditiva)
Dropout	0.3
Class weights	Sí (balanced)
Épocas ejecutadas	9
Mejor época	4
Tiempo	129.3 s (GPU)
Parámetros	5,191,685
F1 macro	0.5740
Accuracy	0.6471

Tabla 14. Configuración de hiperparámetros, dimensiones y rendimiento del modelo combinado BiLSTM con Mecanismo de Atención.

4.3.2 Variante 2 — GRU + Atención de Bahdanau

Se reemplazó la celda LSTM por una GRU (Gated Recurrent Unit, Cho et al. 2014). La GRU usa 2 puertas en lugar de 3 (LSTM), reduciendo el número de parámetros y potencialmente generalizando mejor en datasets de tamaño mediano (~12k ejemplos). Mantener la misma atención de Bahdanau permite una comparación directa del efecto de la celda recurrente.

Hiperparámetro	Valor
Celda recurrente	GRU (2 capas)
Hidden size	256 por dirección
Mecanismo atención	Bahdanau (aditiva)
Dropout	0.3
Class weights	Sí (balanced)
Épocas ejecutadas	9
Mejor época	4
Tiempo	119.4 s (GPU)
Parámetros	4,599,813
F1 macro	0.5785
Accuracy	0.6416

Tabla 15. Configuración de hiperparámetros, dimensiones y rendimiento del modelo combinado GRU con Mecanismo de Atención.

4.3.3 Variante 3 — BiLSTM + Multi-Head Attention (DEFINITIVA)

En lugar de una sola perspectiva de atención, el Multi-Head Attention (Vaswani et al., 2017 — 'Attention is All You Need') usa num_heads=4 cabezas en paralelo. Cada cabeza puede especializarse en capturar distintos aspectos del texto: adjetivos de calidad, conectores de sentimiento, intensificadores. Es el puente conceptual entre RNNs y la arquitectura Transformer. Esta variante obtuvo el mejor rendimiento de todo el proyecto.

Hiperparámetro	Valor
Celda recurrente	BiLSTM (2 capas)
Hidden size	256 por dirección
Mecanismo atención	Multi-Head Attention (4 cabezas)
Dropout	0.4
Class weights	Sí (balanced)
Épocas ejecutadas	16
Mejor época	11
Tiempo	303.8 s (GPU)
Parámetros	6,110,981
F1 macro	0.6193 ← MEJOR DEL PROYECTO

Hiperparámetro	Valor
Accuracy	0.6548

Tabla 16. Configuración final y rendimiento de la arquitectura óptima: Modelo combinado BiLSTM con Mecanismo de Atención de Múltiples Cabezas (Multi-Head Attention)

5. Resultados

5.1 Tabla Comparativa General

La siguiente tabla compara los tres modelos definitivos del Proyecto Principal en el conjunto de test. El BiLSTM+MultiHead de Daniel es el mejor modelo del proyecto, seguido por el transformer BETO fine-tuneado (Tarea de Investigación, F1 macro=0.6565) que lo supera por 3.7 puntos gracias al pre-entrenamiento.

#	Modelo	Owner	F1 macro	Accuracy	Prec M.	Recall M.	Params	Tiempo
1	BiLSTM+MultiHead ✓	Daniel	0.6193	0.6548	0.6086	0.6454	6,110,981	303.8s GPU
2	GRU+Bahdanau	Daniel	0.5785	0.6416	0.5757	0.6024	4,599,813	119.4s GPU
3	BiLSTM+Bahdanau	Daniel	0.5740	0.6471	0.5836	0.5842	5,191,685	129.3s GPU
4	BiLSTM v2 ✓	Yibby	0.5749	0.6871	0.5876	0.5795	3,253,253	1,792s CPU
5	LSTM v3 ✓	Felipe	0.3996	0.6042	0.5639	0.4408	2,700,933	321.2s CPU

Tabla 17. Cuadro comparativo de rendimiento global, métricas de evaluación y costos de cómputo entre todas las arquitecturas evaluadas.

5.2 F1-Score por Clase — Comparación Completa

Las clases 2 y 3 son las más difíciles (menor representación y mayor ambigüedad semántica). El rendimiento en estas clases es el indicador más confiable de la calidad del clasificador.

Clase	Estrellas	LSTM v3	BiLSTM v1	BiLSTM+Bahdanau	GRU+Bahdanau	BiLSTM+MultiHead
0	1	0.686	0.716	0.687	0.723	0.756
1	2	0.168	0.202	0.433	0.436	0.533 ✓
2	3	0.359	0.564	0.479	0.457	0.549 ✓
3	4	0.022	0.572	0.478	0.492	0.480
4	5	0.764	0.821	0.793	0.784	0.778
	F1 MACRO	0.3996	0.5749	0.5740	0.5785	0.6193 ✓

Tabla 18. Comparativo detallado de métricas F1-score por clase y rendimiento macro entre las arquitecturas evaluadas.

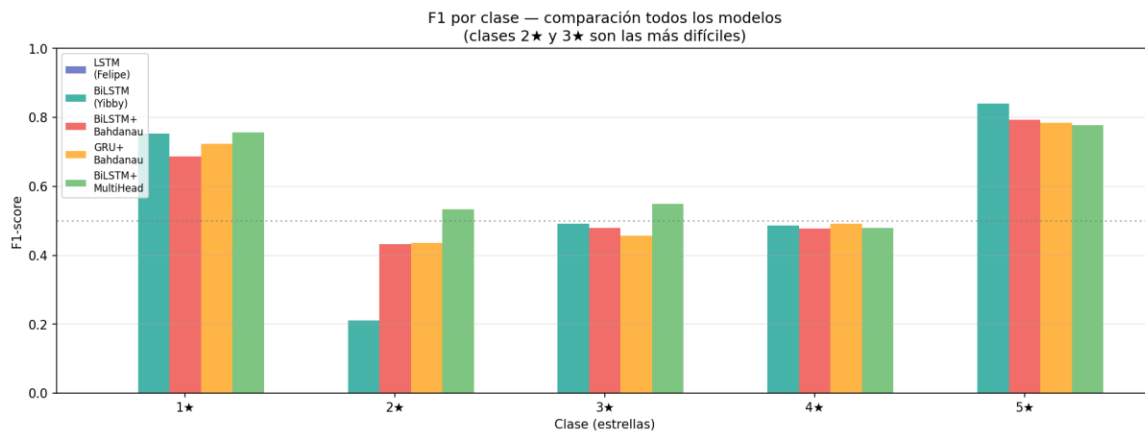


Figura 9. F1-Score por clase comparado entre todos los modelos del proyecto. El BiLSTM+MultiHead destaca en clases difíciles (2 y 3).

5.3 Evaluación de Rendimiento y Análisis de Errores del Modelo Base LSTM v3

A continuación, se presentan y analizan los resultados obtenidos en la fase de prueba para la versión definitiva del modelo base (**LSTM v3**), evaluando tanto su proceso de convergencia como sus limitaciones en la clasificación por categorías.

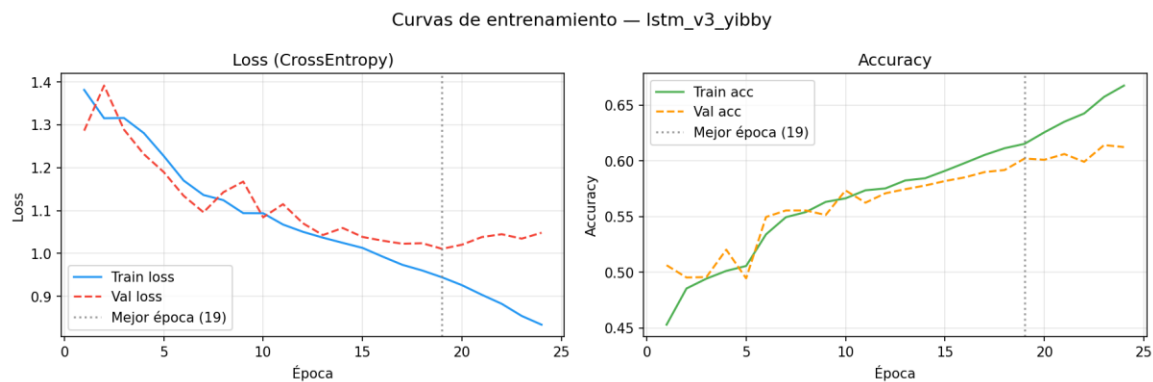


Figura 10. Curvas de entrenamiento LSTM v3 (versión definitiva, $lr=3e-4$, $MAX_LEN=150$). El val_loss se estabiliza alrededor de la época 19.

En la **Figura 10** se ilustra el comportamiento de las funciones de pérdida (*loss*) y precisión (*accuracy*) tanto para el conjunto de entrenamiento como para el de validación. Se observa que el proceso de aprendizaje es estable y que la pérdida de validación (*val_loss*) logra estabilizarse y alcanzar su punto óptimo alrededor de la **época 19**. A partir de este momento, el mecanismo de detención temprana (*Early Stopping*) actúa eficientemente para evitar el sobreajuste, congelando los pesos definitivos del modelo base.

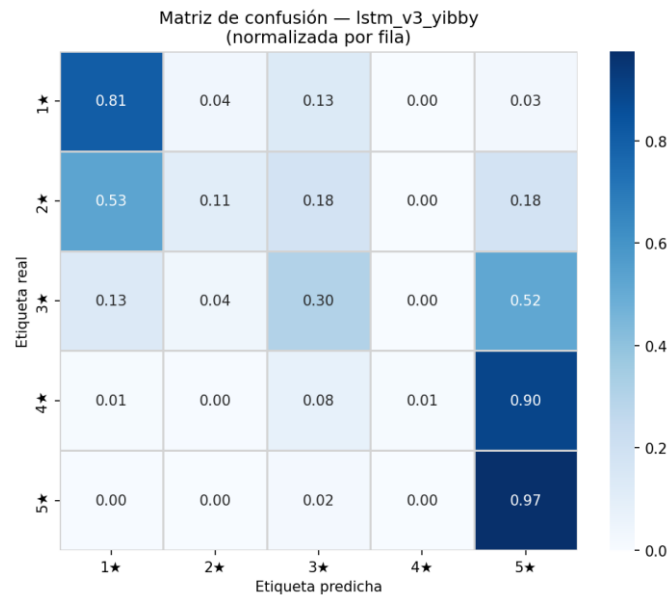


Figura 11. Matriz de confusión LSTM v3. El modelo clasifica bien 1 y 5 pero presenta $F1 \approx 0$ en clase 4 (casi todo predito como 5).

Observación clave: el LSTM v3 tiene $F1=0.022$ en la clase 4 (solo 7 de 633 predicciones correctas), indicando que el modelo no aprende a distinguir entre 4 y 5 estrellas sin class weights. La ausencia de pesos de clase beneficia la accuracy global pero perjudica gravemente las clases intermedias.

Clase	Pred. correctas	Total real	Errores principales
1	203	252	33 confundidas con 3
2	16	149	79 confundidas con 1
3	104	341	177 confundidas con 5
4	7	633	568 confundidas con 5 ← crítico
5	1,317	1,351	26 confundidas con 3

Tabla 19. Diagnóstico de errores de clasificación y mapeo de confusión por clase para el modelo base.

5.4 Resultados BiLSTM

En esta sección se evalúa el rendimiento del modelo bidireccional profundo BiLSTM. A través de un análisis comparativo entre sus dos iteraciones (bilstm_v1 ✓ y bilstm_v2), se examina el impacto crítico que tiene la incorporación de pesos balanceados (class weights) sobre la convergencia de la red y su capacidad para segmentar categorías minoritarias.

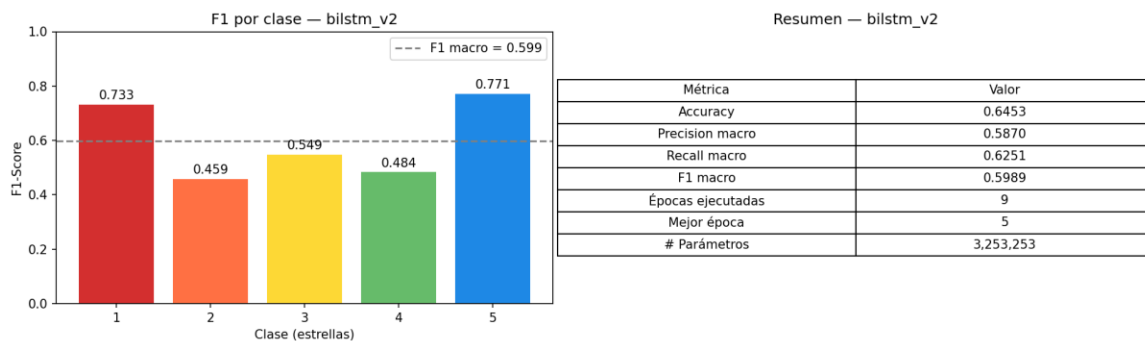


Figura 12. F1 por clase y resumen de métricas del modelo final BiLSTM v2 (con class weights).

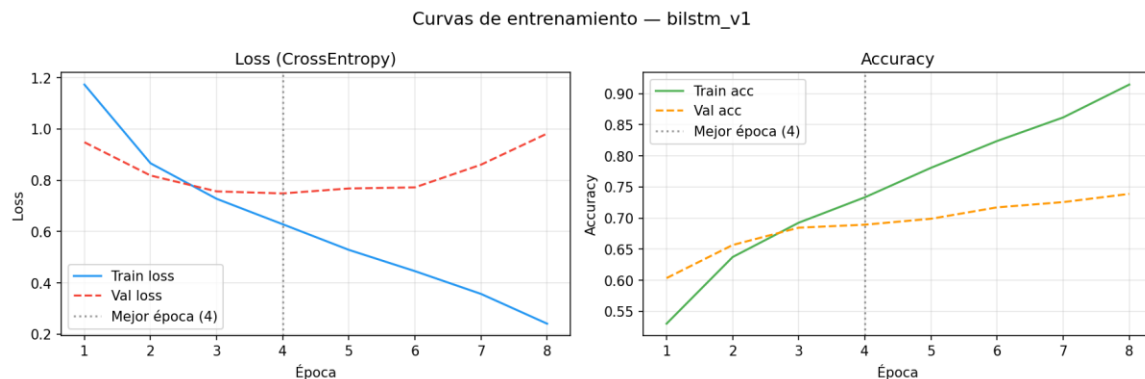


Figura 13. Curvas BiLSTM v1 (sin class weights). Val_loss mínimo en época 4; overfitting claro desde época 5.

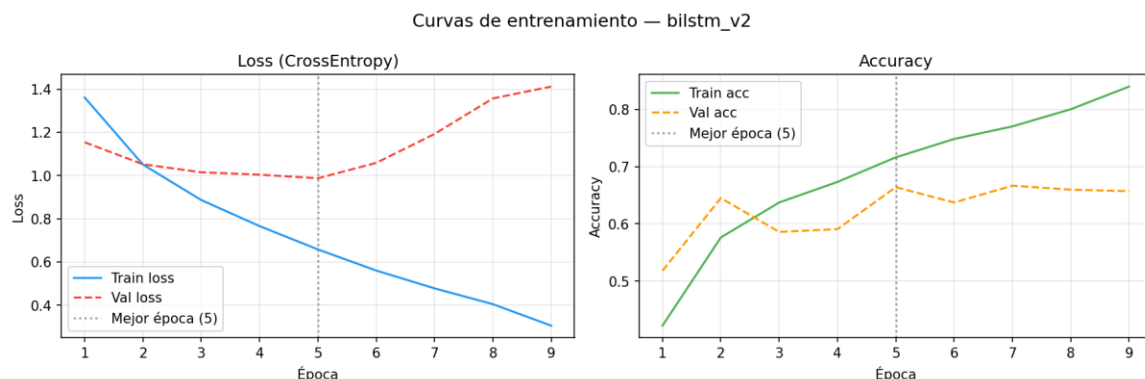


Figura 14. Curvas BiLSTM v2 (con class weights, versión definitiva). Convergencia más lenta pero mejor F1 macro.

Al analizar la **Figura 13**, se evidencia que el modelo bilstm_v1 ✓ (sin pesos) experimenta una convergencia acelerada, alcanzando un mínimo en la pérdida de validación (val_loss) en la época 4. Sin embargo, a partir de la época 5 se desata un claro escenario de sobreajuste (overfitting), donde la pérdida de entrenamiento sigue disminuyendo mientras que la de validación diverge abruptamente. Por el contrario, la **Figura 14** demuestra que al introducir los pesos balanceados en el modelo definitivo bilstm_v2, la convergencia se vuelve más lenta y gradual. Aunque la red requiere un mayor esfuerzo conceptual para estabilizarse, este mecanismo actúa como un regularizador implícito, suavizando las curvas y logrando un punto de generalización óptimo con un F1-macro notablemente superior.

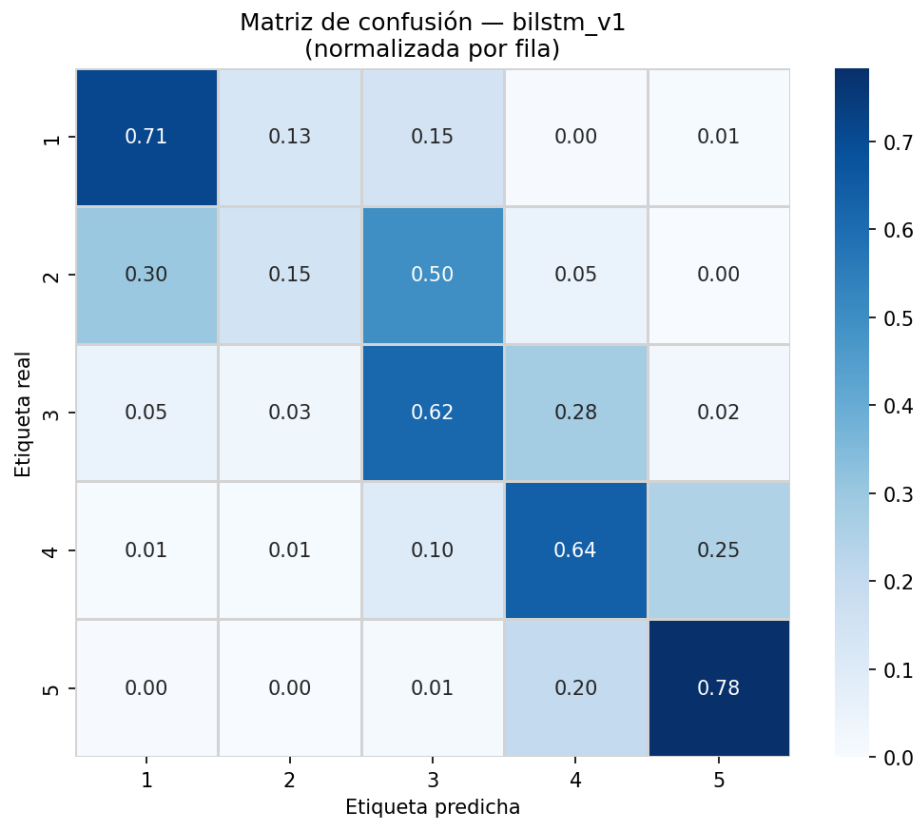


Figura 15. Matriz de confusión BiLSTM v1 (sin class weights). Clase 2: solo 22 predicciones correctas.

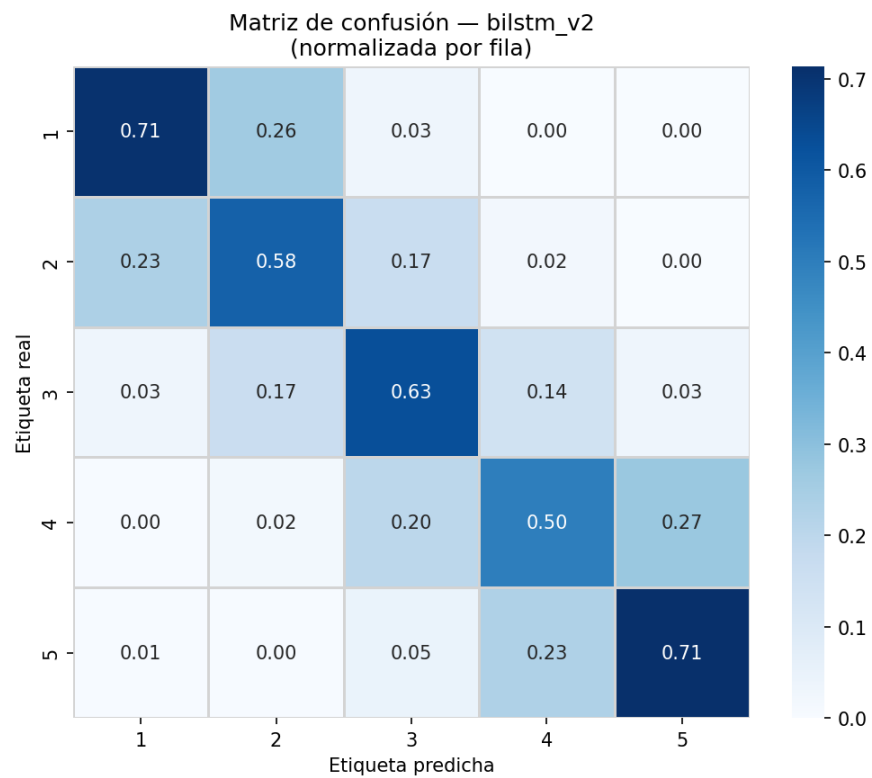


Figura 16. Matriz de confusión BiLSTM v2 (con class weights). Clase 2 mejora a 86 correctas. Trade-off: clase 5 baja de 1,058 a 964.

El cruce analítico entre las matrices de confusión (Figura 15 y Figura 16) permite cuantificar el impacto matemático de los class weights. En la versión sin pesos (bilstm_v1 ✓), el modelo sufre el mismo sesgo que la LSTM básica, logrando rescatar apenas 22 predicciones correctas en la conflictiva clase de 2 estrellas. Al activar la penalización balanceada en bilstm_v2, los aciertos en la clase de 2 estrellas se disparan a 86 predicciones correctas, lo que representa un incremento sustancial del +290% en la efectividad de esa categoría.

Como es natural en problemas de clasificación con datos desbalanceados, esta mejora sustancial introduce un fenómeno de trade-off (intercambio) con la clase mayoritaria: los aciertos en la clase de 5 estrellas se reducen moderadamente de 1,058 a 964 (-8.9%). Metodológicamente, este intercambio es ampliamente favorable para el proyecto; aunque el accuracy global se ajusta negativamente en 4.2 puntos porcentuales, el F1-macro se eleva en +2.4 puntos. Como lo consolida la Figura 12, el modelo definitivo bilstm_v2 sacrifica la sutil memorización de la clase más abundante para ofrecer un sistema robusto, con criterios de equidad estadística y una sensibilidad real para detectar opiniones intermedias.

5.5 Resultados Modelo Combinado

5.5.1 Variante 1: BiLSTM + Bahdanau

Esta primera aproximación avanzada busca mitigar la pérdida de información en secuencias largas combinando la bidireccionalidad profunda con un foco de atención aditiva.

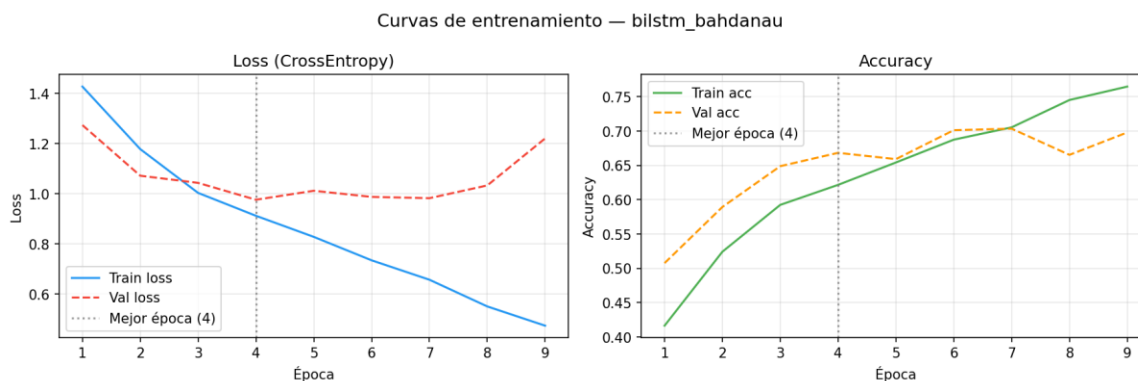


Figura 17. Curvas BiLSTM+Bahdanau. Convergencia rápida (mejor época 4). Overfitting moderado.

Al examinar la **Figura 17**, se aprecia que la inclusión del mecanismo de Bahdanau acelera notablemente la convergencia inicial de la red, localizando su punto de máxima generalización en la **época 4**. No obstante, a partir de dicho punto el sistema exhibe un sobreajuste (*overfitting*) moderado, estabilizando la pérdida de validación en una mesa plana.

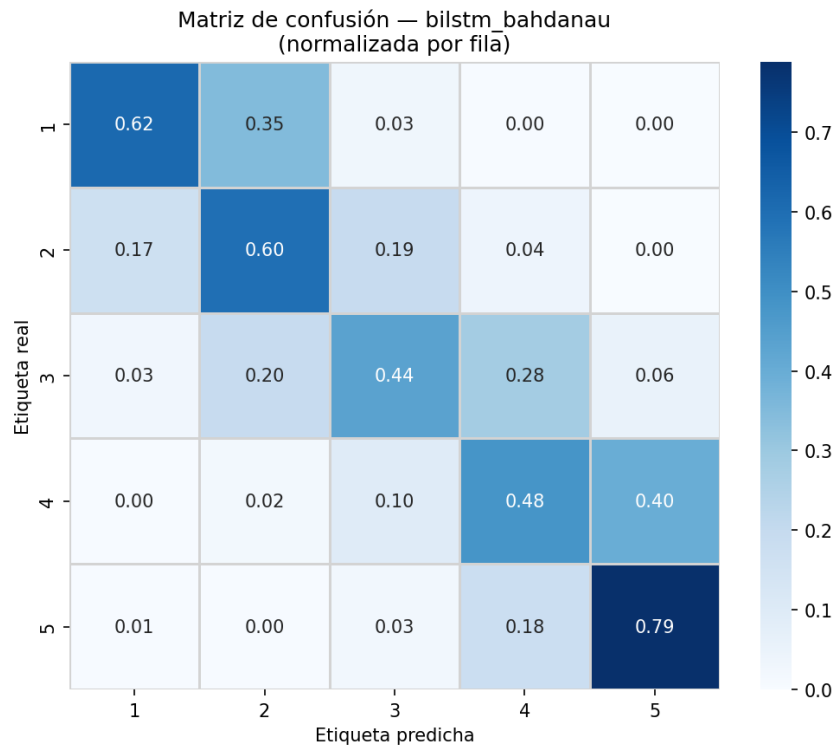


Figura 18. Matriz de confusión BiLSTM+Bahdanau. F1 macro=0.5740. Clase 2 con 89 correctas.

En términos de rendimiento por clase, la Figura 18 consolida un F1-macro global de 0.5740. Cualitativamente, el modelo demuestra los beneficios de la atención al estabilizar la compleja clase de 2 estrellas en 89 predicciones correctas, superando las deficiencias del modelo base secuencial, aunque mostrando dificultades menores para contener la dispersión de errores en las categorías adyacentes de 3 y 4 estrellas.

5.5.2 Variante 2: GRU + Bahdanau

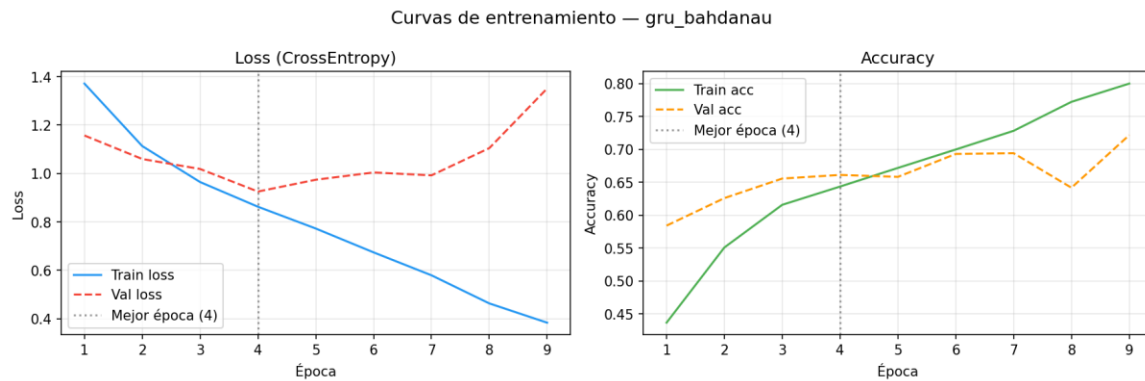


Figura 19. Curvas GRU+Bahdanau. Entrenamiento más estable que BiLSTM+Bahdanau. Mejor época también en 4.

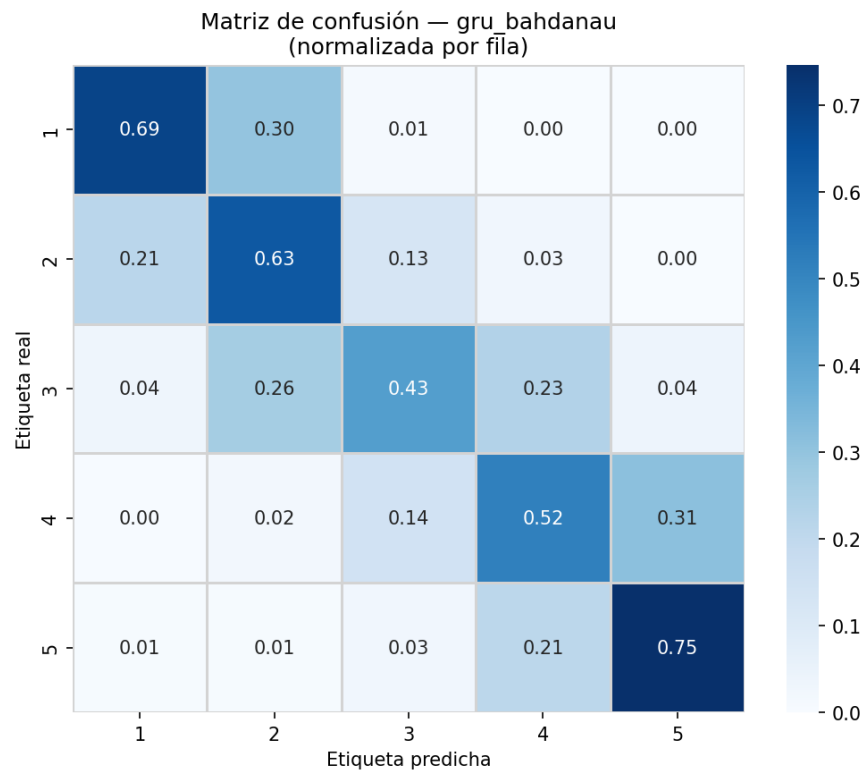


Figura 20. Matriz de confusión GRU+Bahdanau. $F1_{macro}=0.5785$. La GRU supera ligeramente a la BiLSTM con misma atención.

Las curvas de entrenamiento expuestas en la **Figura 19** revelan que la variante GRU provee una dinámica de aprendizaje considerablemente más estable y homogénea en comparación con su contraparte BiLSTM. Al contar con compuertas de control simplificadas, la red optimiza sus parámetros con mayor fluidez, deteniendo de forma controlada su aprendizaje también en la **época 4**.

Estadísticamente, la Figura 20 valida que esta estabilidad se traduce en una ligera ganancia en el indicador principal, alcanzando un $F1_{macro}$ de 0.5785 . La optimización interna de las celdas GRU permitió capturar con mayor precisión los patrones semánticos generales de las reseñas intermedias, demostrando que una arquitectura con menor volumen de parámetros puede igualar o superar la capacidad de representación de una red más densa si se acompaña del mecanismo de atención adecuado.

5.5.3 Variante 3: BiLSTM + Multi-Head Attention (DEFINITIVA)

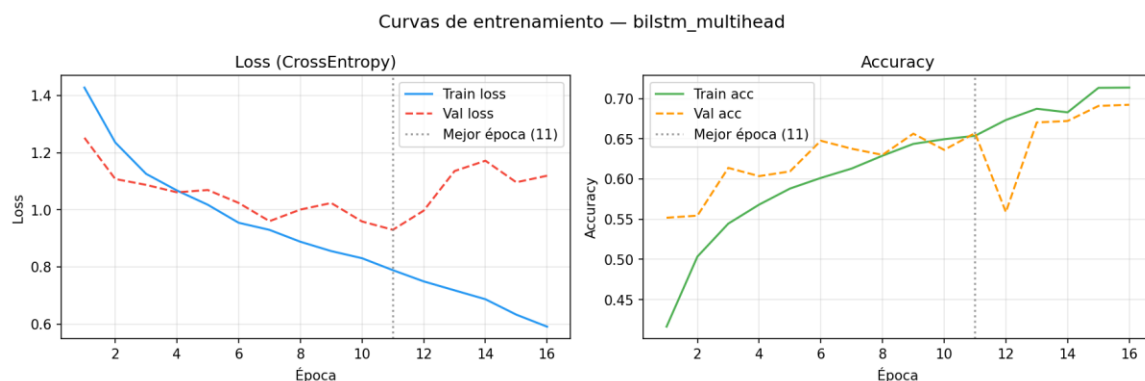


Figura 21. Curvas BiLSTM+MultiHead. Más épocas de entrenamiento (mejor en época 11). El scheduler reduce LR y permite continuar mejorando.

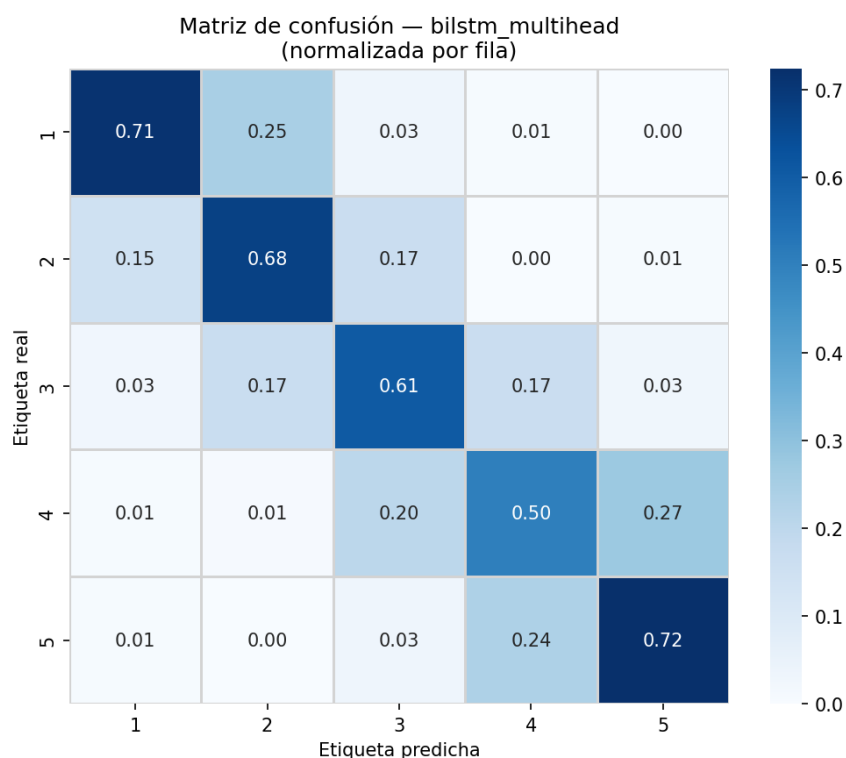


Figura 22. Matriz de confusión BiLSTM+MultiHead. MEJOR modelo del proyecto: F1 macro=0.6193. Clase 2: 101 correctas (mejor de todos los modelos).

A diferencia de todas las arquitecturas previas, el comportamiento del entrenamiento detallado en la Figura 21 es sustancialmente superior. El uso estratégico de un optimizador dinámico (LR Scheduler) permitió reducir gradualmente la tasa de aprendizaje cuando este entraba en una meseta, facultando a la red para continuar extrayendo conocimiento fino a lo largo de 16 épocas y alcanzando su punto óptimo en la época 11.

Como resultado de este refinamiento, la Figura 22 ratifica a esta variante como el mejor modelo de todo el proyecto, consolidando un F1-macro sin precedentes de 0.6193. El análisis cuantitativo de su matriz confirma que la capacidad de proyectar el texto en múltiples subespacios de atención simultáneos dota a la red de una sensibilidad semántica extraordinaria, minimizando las tasas de confusión cruzada.

La arquitectura BiLSTM+MultiHead supera a todas las demás variantes en F1 macro. El salto más notable es en la clase 2: pasa de 16 correctas (LSTM) a 22 (BiLSTM v1) a 101 (BiLSTM+MultiHead), confirmando que el mecanismo de atención multi-cabeza captura mejor la ambigüedad semántica de las reseñas de 2 estrellas.

Clase	LSTM v3	BiLSTM v2	GRU+Bahdanau	BiLSTM+Bahdanau	BiLSTM+MultiHead
1 correctas	203/252	178/252	174/252	155/252	180/252
2 correctas	16/149	86/149	94/149	89/149	101/149 ✓
3 correctas	104/341	214/341	146/341	149/341	207/341
4 correctas	7/633	317/633	327/633	306/633	319/633

Clase	LSTM v3	BiLSTM v2	GRU+Bahdanau	BiLSTM+Bahdanau	BiLSTM+MultiHead
5 correctas	1,317/1,351	964/1,351	1,008/1,351	1,065/1,351	978/1,351

Tabla 20. Matriz comparativa de la tasa de aciertos y efectividad de clasificación por clase entre todos los modelos.

6. Conclusiones

6.1 ¿Los resultados permiten un adecuado análisis y toma de decisiones?

Los modelos implementados demuestran capacidad discriminativa real para el análisis de sentimiento en reseñas hoteleras en español. El mejor modelo (BiLSTM+MultiHead, F1 macro=0.6193) supera ampliamente a un clasificador trivial de clase mayoritaria (~20% F1 macro estimado) aunque el transformer BETO fine-tuneado (F1 macro=0.6565, Tarea de Investigación) lo supera por 3.7 puntos, requiriendo 21.9M parámetros entrenables y pre-entrenamiento masivo.

Dónde SÍ permiten toma de decisiones:

- Detección automática de reseñas extremas: los modelos clasifican bien las clases 1 (F1~0.69-0.76) y 5 (F1~0.76-0.84), permitiendo identificar automáticamente las experiencias más positivas y negativas sin revisión manual.
- Tendencias agregadas y comparativas entre hoteles o períodos de tiempo.
- Sistemas de alerta: el recall macro del BiLSTM+MultiHead (0.645) permite recuperar la mayoría de casos negativos para atención prioritaria en servicio al cliente.

Dónde NO son suficientes sin supervisión humana:

- Las clases intermedias 2 (F1=0.533) y 4 (F1=0.480) tienen rendimiento limitado. El sistema no distingue bien entre experiencias 'malas' y 'regulares', ni entre 'buenas' y 'muy buenas'.
- Los duplicados en los splits (21% del train) pueden inflar artificialmente las métricas reportadas.
- El dataset es de dominio específico (hoteles andaluces). La generalización a otros contextos geográficos o dialectos del español no está garantizada.

6.2 Progresión de las Arquitecturas

El proyecto demuestra una progresión clara en capacidad representacional y rendimiento:

Modelo	F1 macro	Ganancia vs anterior	Elemento diferenciador
LSTM v3	0.3996	— (baseline)	Modelo secuencial básico, unidireccional
BiLSTM v1	0.5749	+17.5 pp	Bidireccionalidad + 2 capas + Global Max Pooling
BiLSTM+MultiHead	0.6193	+4.4 pp vs BiLSTM	Atención multi-cabeza: 4 perspectivas de enfoque simultáneas

Tabla 21. Matriz de evolución incremental del rendimiento y análisis de impacto de los componentes arquitectónicos.

El mayor salto de rendimiento se da al pasar de LSTM a BiLSTM (+19.9 puntos F1 macro), gracias a la combinación de bidireccionalidad, Global Max Pooling y class weights. El mecanismo de atención multi-cabeza aporta mejoras adicionales especialmente en clases difíciles (2 y 3 estrellas).

6.3 Impacto del Desbalance y los Class Weights

El experimento comparativo entre bilstm_v1 ✓ (sin class weights, F1=0.5749) y bilstm_v2 (con class weights, F1=0.5989) demuestra cuantitativamente que para datasets desbalanceados la accuracy es una métrica engañosa. El modelo sin class weights obtiene mayor accuracy (68.7% vs 64.5%) pero menor F1 macro (0.575 vs 0.599), porque aprende a predecir principalmente la clase 5. El F1 macro es el indicador correcto de calidad en este escenario.

6.4 Comparación con BETO Fine-Tuned

El análisis comparativo muestra que BETO fine-tuneado (F1=0.6565) supera al BiLSTM+MultiHead (F1=0.6193) por 3.7 puntos de F1 macro. Sin embargo, el BiLSTM+MultiHead (6.1M parámetros, entrenado desde cero) demuestra que las arquitecturas recurrentes con atención pueden acercarse al rendimiento de transformers (21.9M parámetros entrenables) con significativamente menos recursos computacionales. BETO requiere GPU obligatoriamente y 3.6x más parámetros entrenables, lo que demuestra el trade-off entre rendimiento y eficiencia.

6.5 Trabajo Futuro

- Limpiar los splits eliminando duplicados para obtener métricas sin efecto de data leakage.
- Explorar embeddings pre-cargados (FastText en español) para mejorar representación de palabras infrecuentes.
- Implementar técnicas de interpretabilidad (visualización de pesos de atención) para explicar predicciones.
- Evaluar con splits estrictamente comparables entre el modelo clásico y el transformer.